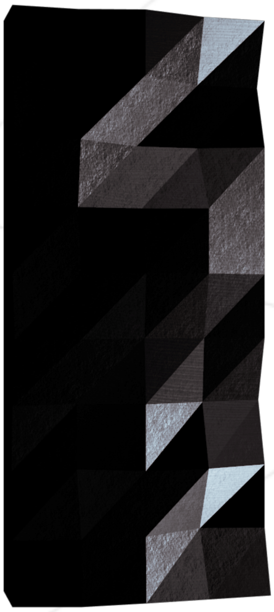

SCOP

Introduction to GPU rendering

Summary: This small project is a first step toward GPU rendering using OpenGL, Vulkan or Metal APIs.

Version: 5.0

Contents

Chapter I

Foreword

It's simply because we've never done that one yet!

Chapter II

AI Instructions

Context

AI is now a powerful coding partner — alongside your peers — for tackling large and demanding projects. You will guide it through both technical and non-technical aspects of your work.

AI tools can boost your efficiency and improve the quality of your output, but you should be able to dive deep into any part of the project without relying on them.

Your AI partner supports you, but you remain fully responsible for making informed technical decisions and to clearly explain and defend them.

Main message

☛ Strive for a mature and responsible use of AI.

☛ Never let AI take responsibility for decisions — especially when it lacks awareness of your goals, constraints, or team dynamics.

☛ Maintain creativity, innovation, and human oversight through active collaboration with your peers. AI is trained on existing data and rarely generates truly new ideas.

☛ Stay informed about emerging trends and be ready to adapt to new concepts and technologies.

Learner rules:

- Maintain intellectual leadership over your projects and make your own informed decisions.
- Prioritise the collective intelligence of your team and peers.
- Actively stay informed about the ongoing evolution of AI technologies.

Phase outcomes:

- AI engineering skills.
- Increased efficiency.
- Greater reliability and quality.
- A pioneering mindset.

Comments and examples:

- Your peers can identify trade-offs, question assumptions, and help you improve. The first answer from an AI might not be the best — it may lack efficiency, security, or real added value. Now more than ever, you should rely on your peers.
- AI can make you faster, but your peers make you better. Collaboration, discussion, and mutual challenge are key to success.
- Be transparent about how AI was used in your projects, and clearly identify what was generated by AI tools.

✓ Good practice:

> I asked AI to help generate unit tests for my API. I reviewed them with my teammate, and we adjusted them for edge cases. It saved time, and we both learned something new.

✗ Bad practice:

> I had AI generate the entire architecture of my project. It "works," but when I'm asked to explain design decisions during the peer review or in front from of a customer, I cannot. I lose credibility and I fail.

Chapter III

Subject

III.1 Feeling good doesn't hurt.

Sometimes, it just feels good to feel good. So let's build a small application that makes us feel great.

Moreover, this project will give you a solid intellectual workout. As such, a few constraints have been introduced to keep things interesting.

III.2 What You Need to Do

Your mission is to build a small program that displays a 3D object created with a modeling tool such as Blender.

This 3D object will be stored in a `.obj` file. You will be responsible for parsing it and rendering it correctly.

Inside a window, the 3D object must be displayed in perspective (i.e., objects farther away appear smaller), and you must be able to rotate and translate it around its three main axes of symmetry (with the origin at the center of the object), using any user input method of your choice (keyboard, mouse, etc.).

Using various colors, you must make the different faces of the object visually distinguishable (either a 3D face as defined in the `.obj` file, or a triangle if you choose to triangulate the model).

Finally, pressing a dedicated key (or using any other user input method) should apply a texture to the object. Pressing that same key again should toggle it back to the colored view. A smooth transition between the two is expected, with no abrupt cuts.

Here are the technical constraints:

- You are free to use any language (C, C++, or Rust preferred).
- You can choose between OpenGL, Vulkan, or Metal.
- Provide a one-line command to check for and possibly install all required dependencies to run the project (e.g., a classic Makefile for a C project).
- External libraries are permitted only for window and event management.
- No external libraries are permitted for loading the 3D object, creating matrices, or loading shaders: you will have to implement these yourself using the OpenGL, Vulkan, or Metal API.

Since this project is something of a self-congratulatory exercise, it is essential that, during the defense, you showcase the 42 logo (provided in the resources), spinning around its central axis (not around one of its corners!).

The faces should feature subtle shades of gray (in keeping with the spirit of the 42 logo), and the texture should depict something cheerful, such as ponies, kittens, or unicorns (your choice).

```
During the defense, expect to test additional 3D objects too.
```

```
The 42 logo is copyrighted, and can be used in this project only for
pedagogical purposes.  Do not use it outside of the context of this
project.
```

Chapter IV

Readme Requirements

A `README.md` file must be provided at the root of your Git repository. Its purpose is to allow anyone unfamiliar with the project (peers, staff, recruiters, etc.) to quickly understand what the project is about, how to run it, and where to find more information on the topic.
The `README.md` must include at least:

- The very first line must be italicized and read: *This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]].*
- A "**Description**" section that clearly presents the project, including its goal and a brief overview.
- An "**Instructions**" section containing any relevant information about compilation, installation, and/or execution.
- A "**Resources**" section listing classic references related to the topic (documentation, articles, tutorials, etc.), as well as a description of how AI was used — specifying for which tasks and which parts of the project.

Additional sections may be required depending on the project (e.g., usage examples, feature list, technical choices, etc.).

Any required additions will be explicitly listed below.

Chapter V

Bonus

Here are the possible bonus features:

- *[10pt]* Proper handling of complex `.obj` files, such as concave or non-coplanar ones. The teapot included in the resources comes in two versions: one is the original, which exhibits some unusual edge artifacts; the other has been re-exported from Blender (without manual adjustments) and slightly normalized by the software. Try to render the first one correctly.
- *[5pt]* Apply textures more subtly, avoiding visible stretching on certain faces!
- *[10pt]* We are confident that you can think of even more impressive features to add.

The bonus section will only be evaluated if the mandatory part is PERFECT. "Perfect" means every single requirement has been completed and works flawlessly. If you miss even one mandatory feature, the bonus won't be considered at all.

Chapter VI

Submission and Peer-Evaluation

Submit your work in your `Git` repository as usual. Only what is in the repository will be reviewed during the defense.

Chapter VII

Gallery

Figure VII.1: The 42 logo with shades of gray on its faces

Figure VII.2: From the back, with a texture

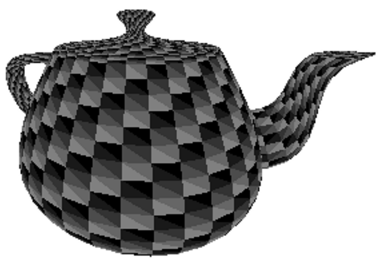

Figure VII.3: The teapot